

University of Victoria

Department of Electrical and Computer Engineering
ECE 455 - Real Time Computer Systems Design Project

Laboratory Project
Deadline Driven Scheduler

Date of Demonstration: March 23, 2026

Report Submitted on: March 30th, 2026

To: Dr. A. Baniasadi

Muhammad Wali-ur-Rehman V01005929
Ben Towers V00978554

Introduction.....	3
Design Solution.....	3
Overall Architecture.....	3
List of F-Tasks and Priority Levels.....	4
Use of FreeRTOS Features.....	5
Task Roles and Responsibilities.....	5
Core Data Structures.....	6
DDS Public Interface and Internal Behavior.....	6
DD-Task Sorting Algorithm.....	7
Preemption Mechanism.....	9
Deadline Detection and Overdue Classification.....	9
Generator Implementation.....	9
User Task Implementation.....	10
Monitor Implementation.....	10
Benchmark Configuration and Workload Calibration.....	10
End-to-End Flow of a Job.....	11
Discussion.....	11
Table of DDS Events Generated for Test Bench 1.....	11
Table of Monitor Task Outputs for Test Bench 2.....	13
Discussion of DDS Scheduler Performance.....	13
Limitations and Possible Improvements.....	13
Summary.....	14
Appendices.....	14
Appendix A: Code.....	14
Appendix B: Design Document.....	49

Introduction

This project implements a Deadline-Driven Scheduler (DDS) on top of FreeRTOS for the STM32F4 Discovery board. The objective is to schedule deadline-sensitive tasks using Earliest Deadline First (EDF) while still relying on the native FreeRTOS kernel for actual context switching and task execution.

In this design, a deadline-driven task is represented by a DDS-managed metadata record rather than by a custom kernel object. Each DDS task corresponds to a normal FreeRTOS user task whose priority is adjusted dynamically by the DDS. The task with the earliest absolute deadline is promoted to a higher user-task priority, while all other active user tasks remain at a lower priority. This lets the standard FreeRTOS preemptive scheduler enforce EDF behavior indirectly.

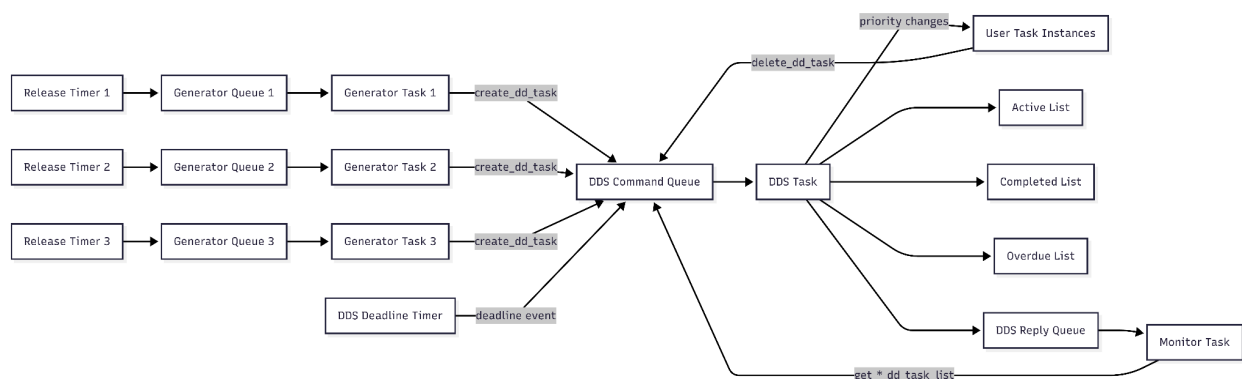
The system also includes periodic task generators, a monitor task, and a benchmark configuration layer. These support the required test benches and provide a clean structure for analysis, debugging, and extension.

Design Solution

Overall Architecture

The system is composed of a small set of interacting runtime components. The DDS task is the central scheduling authority. Generator tasks create new jobs periodically according to the selected test bench. User tasks execute the calibrated workload associated with each released job. The monitor task requests snapshots of the scheduler state and was used to validate behavior against expected counts. Software timers are used to trigger periodic job releases, monitor sampling, and deadline-expiry events.

The most important design decision is that the DDS is the only owner of scheduler state. Other tasks do not directly manipulate the active, completed, or overdue lists. Instead, they communicate with the DDS only through message-passing interfaces. This follows the project requirements and also prevents state corruption caused by multiple modules trying to update the scheduler concurrently.



This architecture lets the DDS behave like a small event-driven scheduling server. Release events, completion events, monitor requests, and deadline events all flow into one command queue. The DDS drains that queue, updates its internal lists, reapplies EDF priorities, and then suspends itself until the next event arrives.

List of F-Tasks and Priority Levels

The following FreeRTOS tasks are used in the design. Some are permanent tasks created at startup, while others, such as user task instances, are created dynamically at release time.

F-Task	Purpose	Priority
DDS	Owns DDS state and applies EDF scheduling	configMAX_PRIORITIES - 1
Tmr Svc	FreeRTOS timer service task	configMAX_PRIORITIES - 2
GEN1	Generator for benchmark task source 1	configMAX_PRIORITIES - 3
GEN2	Generator for benchmark task source 2	configMAX_PRIORITIES - 3
GEN3	Generator for benchmark task source 3	configMAX_PRIORITIES - 3
MON	Monitor task for DDS state observation	configMAX_PRIORITIES - 3
USR1 instances	Dynamic user-task releases for source 1	configMAX_PRIORITIES - 4 when selected, otherwise 1
USR2 instances	Dynamic user-task releases for source 2	configMAX_PRIORITIES - 4 when selected, otherwise 1
USR3 instances	Dynamic user-task releases for source 3	configMAX_PRIORITIES - 4 when selected, otherwise 1
IDLE	Standard FreeRTOS idle task	lowest

This priority arrangement ensures that the DDS always responds immediately to new events, that timer callbacks are not starved by user computation, and that exactly one active user task can outrank the others at a time.

Use of FreeRTOS Features

The design makes direct use of the FreeRTOS features emphasized in the course. Tasks are used to represent the DDS, generators, monitor, and per-release user workloads. Queues are used for all inter-task communication involving scheduler events, monitor queries, and generator wake tokens. Software timers are used both for periodic release behavior and for DDS deadline checking. Dynamic task creation and deletion are used so that each released DD-task corresponds to a unique FreeRTOS task instance, which keeps the implementation easy to reason about and avoids state leakage between releases of the same logical task.

Task notifications were intentionally not used, because the project requirements explicitly prohibit them. Global variables are also not used for inter-task communication. The only global scheduler state is private to the DDS implementation and is never accessed directly by auxiliary tasks.

Task Roles and Responsibilities

The DDS task is the most important task in the system. It runs at the highest application priority and is normally suspended. When resumed, it drains all pending scheduler messages, performs any needed state transitions, applies EDF ordering, and rearms the one-shot deadline timer to the earliest active deadline. Because it always runs at the highest application priority, once an event wakes the DDS it can immediately restore the correct scheduling state before any user task continues execution.

The generator subsystem is built from three dedicated generator tasks, one for each periodic source task in the benchmark set. Each generator has its own queue and software timer. The timer callback does only a small amount of work: it captures the current tick count, sends a release token into the generator's queue, and resumes the generator task. The generator task then creates a new suspended user task, computes the absolute deadline for that release, and registers the new job with the DDS.

User tasks represent the actual deadline-sensitive application code. In this project they do not perform a real application calculation, but instead execute a calibrated busy-work loop. The user task turns on a source-specific LED for visual debugging, performs its workload, reports completion to the DDS using `delete_dd_task(task_id)`, and then suspends itself so that the DDS can safely delete the task handle.

The monitor task exists to collect scheduler state using the required getter functions. It requests cloned snapshots of the active, completed, and overdue lists and can print concise summary lines. During development, monitor printing was useful for understanding system behavior, but because cloned list snapshots and SWV output add nontrivial overhead, printing was later commented out for the tightest timing tests.

Core Data Structures

The shared scheduler data model is defined in `src/dds_types.h`. The central structure is `dd_task_t`, which stores the FreeRTOS task handle, task type, task ID, release time, absolute deadline, completion time, execution budget in milliseconds, source task number, sequence number, and current DDS state. This structure is not a task itself. It is a metadata record that tells the DDS how to reason about a particular released job.

The scheduler lists are implemented as singly linked lists of `dd_task_node_t`, where each node contains one `dd_task_t` and a pointer to the next node. This structure is simple, efficient for small task counts, and easy to verify. Since the project does not require large-scale scheduling, a linked list is entirely adequate and keeps the code easier to understand than a more complex data structure.

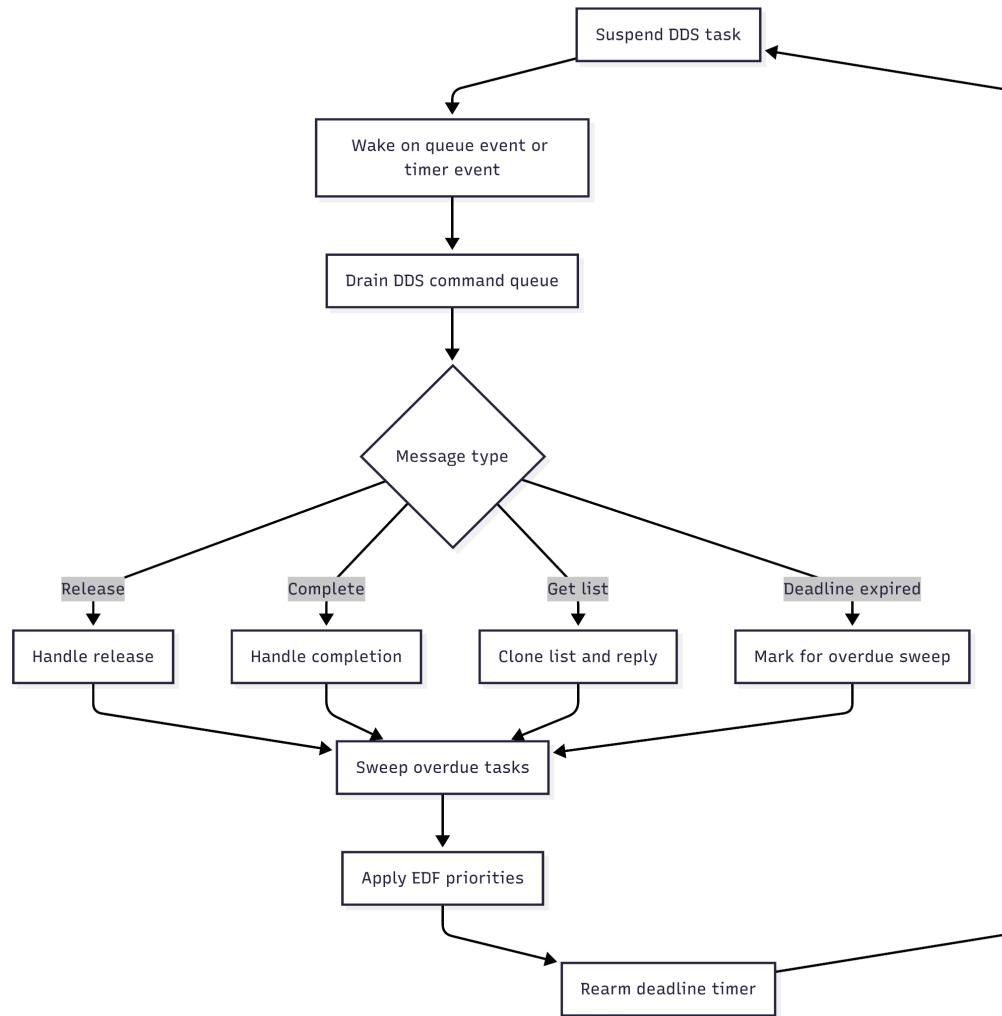
Messages sent to the DDS are represented by `dds_message_t`. Each message includes a type field and a union payload. Release messages carry an entire task record, while completion messages carry only the task ID and the observed completion time. This compact message-based interface keeps all DDS state transitions serialized through one queue.

DDS Public Interface and Internal Behavior

The public DDS API consists of `create_dd_task(...)`, `delete_dd_task(...)`, and the three getter functions for the active, completed, and overdue lists. These functions are intentionally thin wrappers. They package a message, send it to the DDS command queue, and resume the DDS task. None of them directly manipulates the scheduler lists.

Inside `src/dds.c`, the DDS task waits suspended until one of these interface calls wakes it. When resumed, it drains all queued messages before returning to sleep. This detail is important because it means multiple releases or completions that arrive close together are handled as one burst of scheduler work. After all queued messages are processed, the DDS performs an overdue sweep, reapplies EDF priorities, and rearms the one-shot deadline timer.

The main behavior of the DDS can be summarized as follows:

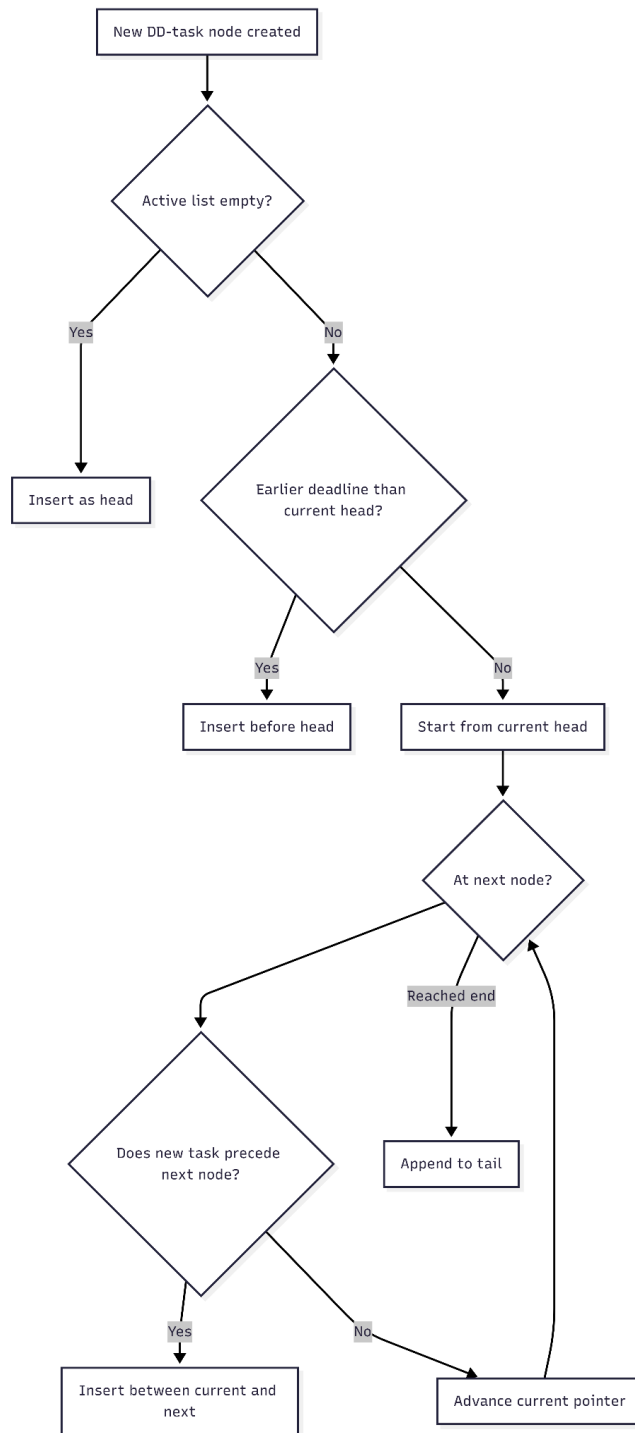


DD-Task Sorting Algorithm

The active, completed, and overdue lists are managed by the helper functions in `src/dds_list.c`. The active list is the only list that must remain sorted. It is maintained using sorted insertion rather than by inserting unsorted nodes and then sorting the entire list afterward. This was chosen because the number of active tasks is small and because sorted insertion is easy to reason about and verify.

The ordering rule is deterministic. A task precedes another if it has an earlier absolute deadline. If the deadlines are equal, the task with the earlier release time comes first. If the release times are also equal, the smaller task ID comes first. This creates a stable EDF order and removes ambiguity when multiple tasks share the same deadline.

The flow chart below shows how the DD-task sorting logic works during insertion into the active list.



The completed and overdue lists do not need to be sorted, so tasks are appended to the tail in the order they are classified. Those lists therefore behave as historical logs rather than active scheduling structures.

Preemption Mechanism

Preemption is not implemented by a custom context switch routine. Instead, it is implemented indirectly through FreeRTOS priority changes. Whenever the active list changes, the DDS walks that list and assigns `DDS_PRIO_USER_HIGH` to the head and `DDS_PRIO_USER_LOW` to every other active task. It then resumes the selected head task. If a new task arrives with an earlier deadline than the currently running task, the current task is demoted and the new task is promoted. Because FreeRTOS is preemptive, that promotion causes the newly selected task to preempt the old one.

This design preserves the requirement that normal code execution is task-based, while still giving the DDS precise control over which task should be running according to EDF.

Deadline Detection and Overdue Classification

The system uses a one-shot DDS-owned deadline timer to catch the earliest active deadline. After each scheduler update, the DDS arms that timer to the absolute deadline of the active-list head. When the timer expires, it sends a deadline-expired message back to the DDS command queue and resumes the DDS task. The callback itself remains short and does no heavy work.

Overdue classification is then performed inside the DDS task by sweeping the active list head while the current time is greater than the head task's deadline. This works because the active list is sorted; if the earliest deadline task is not late, no later task can already be overdue.

Completion classification follows a similar idea, but instead of using the DDS service time it uses an observed completion timestamp captured inside `delete_dd_task(...)`. This design choice was important because it prevents a task from being marked late simply because DDS handled its completion message slightly after the actual finish time.

Generator Implementation

The generator implementation in `src/generator.c` uses one dedicated generator task per source rather than one generalized generator for all tasks. This approach maps cleanly to the benchmark structure, keeps the code easy to debug, and makes the timer-to-generator relationship explicit. Each generator task has a sequence counter that increments once per release token. The task ID is then encoded as the source number in the upper 16 bits and the sequence number in the lower 16 bits.

When a generator processes a release token, it allocates a `user_task_params_t` block, fills it with the task ID, execution time, source number, and calibrated work-iteration count, and creates a new user task. The task is immediately suspended so that it cannot run before the DDS has a chance to incorporate it into the active list. After computing the absolute deadline, the generator calls `create_dd_task(...)` and the DDS takes over from there.

Using a release queue for each generator is important because resume alone is not a counting event. If multiple timer callbacks occur before a generator suspends itself again, queued tokens preserve every release event.

User Task Implementation

The user-task logic in `src/user_task.c` is minimal. After casting its `pvParameters` pointer back to `user_task_params_t`, the task copies the values it needs and frees the parameter block immediately. This avoids leaking one parameter block per released job. The task then turns on the LED corresponding to its source number, runs `busy_work_for_budget(work_iterations)`, turns the LED off, and calls `delete_dd_task(task_id)`. Finally, it suspends itself so that the DDS can safely delete the FreeRTOS task once the completion message has been processed.

The busy-work loop is deliberately artificial. It updates a volatile sink value and executes `__NOP()` inside a counted loop so that the compiler cannot optimize the work away completely. This gives a repeatable approximation of CPU work without depending on wall-clock delays, which would be misleading under preemption.

Monitor Implementation

The monitor logic in `src/monitor.c` uses a software timer and a small event queue. Each time the timer expires, the callback enqueues a token and resumes the monitor task. The monitor then drains any pending tokens, requests the active, completed, and overdue lists through the DDS getter functions, counts their nodes, and frees the clones afterward.

This design is correct from the perspective of encapsulation, because the monitor never sees the actual DDS list pointers. However, the cost of cloning and freeing three lists per monitor sample is real. In addition, SWV output through `printf` adds overhead that is small in absolute terms but meaningful in tight real-time benchmarks.

Benchmark Configuration and Workload Calibration

The benchmark layer in `src/testbench.c` and `src/testbench.h` provides the three required task sets and converts abstract execution times into busy-loop iteration counts. The currently supported task sets are shown below.

Test Bench	Task 1	Task 2	Task 3	Hyperperiod
TB1	95 ms / 500 ms	150 ms / 500 ms	250 ms / 750 ms	1500 ms
TB2	95 ms / 250 ms	150 ms / 500 ms	250 ms / 750 ms	1500 ms
TB3	100 ms / 500 ms	200 ms / 500 ms	200 ms / 500 ms	500 ms

During calibration, the cycles-per-iteration estimate was tuned so that the busy-work loop on the actual board closely matched the intended benchmark execution times. A value of 21 cycles per iteration produced close agreement with the target lengths and was retained in the implementation.

End-to-End Flow of a Job

The lifecycle of one released job captures how the whole system fits together. A software timer fires and its callback sends a release token to the appropriate generator queue. The generator wakes, creates a suspended user task, computes that task's deadline, and calls `create_dd_task(...)`. The DDS receives the release message, stamps the release time, inserts the new job into the active list in EDF order, and reapplies priorities. The selected user task then runs. When it finishes its workload, it calls `delete_dd_task(task_id)`. DDS receives that completion message, compares the observed completion time against the deadline, moves the task into either the completed or overdue list, deletes the underlying FreeRTOS task handle, and then selects the next active job.

Discussion

This section evaluates the performance of the implemented Deadline-Driven Scheduler using the required benchmark test cases. The measured system behavior is compared against the expected Earliest Deadline First (EDF) schedule in order to assess event ordering, timing accuracy, deadline handling, and overall scheduler correctness. The results show that the scheduler behaves correctly in both schedulable and mildly overloaded cases, while also highlighting the effect of implementation overhead on real embedded hardware.

Table of DDS Events Generated for Test Bench 1

Test Bench 1 is schedulable under EDF, so the benchmark is primarily used to verify event order, release timing, and completion timing. The expected event sequence over one hyperperiod is shown below.

Event #	DDS Event for TB1	Expected Time (ms)	Measured Time (ms)
1	Task 1 released	0	0
2	Task 2 released	0	0
3	Task 3 released	0	0
4	Task 1 completed	95	95
5	Task 2 completed	245	245
6	Task 3 completed	495	495
7	Task 1 released	500	500
8	Task 2 released	500	500
9	Task 1 completed	595	595
10	Task 2 completed	745	747
11	Task 3 released	750	750
12	Task 1 released	1000	1000
13	Task 2 released	1000	1000
14	Task 3 completed	1000	1000
15	Task 1 completed	1095	1096
16	Task 2 completed	1245	1247
17	Task 1 released	1500	1500
18	Task 2 released	1500	1500
19	Task 3 released	1500	1500

In our measured runs, the completion and release timing stayed very close to these ideal values, with only about 1 or 2 ms of aggregate delay over the hyperperiod. That small offset is consistent with queue handling, timer latency, tick granularity, and normal implementation overhead rather than with any scheduling error.

Table of Monitor Task Outputs for Test Bench 2

Test Bench 2 is slightly overloaded. Its total demand over one 1500 ms hyperperiod is 1520 ms, so one missed deadline is expected. The measured and expected monitor results are shown below.

Quantity	Measured at 1500 ms
Number of active DD-tasks	3
Number of completed DD-tasks	10
Number of overdue DD-tasks	1

This correctly reflects the output we expected where one task is labeled overdue at the boundary as it can not be completed before the end of the hyperperiod. The three active tasks are the new tasks created for the next hyperperiod.

Discussion of DDS Scheduler Performance

From a scheduler-performance perspective, the implementation behaved well in the schedulable and mildly overloaded cases. In Test Bench 1, EDF ordering was stable and repeatable, the active-task count evolved as expected, and no overdue tasks appeared. In Test Bench 2, the scheduler exhibited the correct overload behavior by allowing one job to become overdue rather than producing chaotic misordering or runaway backlog. This indicates that the scheduler logic itself is sound and that task-state transitions are being handled correctly.

The main performance cost comes from support infrastructure rather than from the EDF logic itself. Dynamic task creation and deletion, cloned monitor list snapshots, SWV printing, queue traffic, and timer callbacks all consume CPU time. In Test Bench 1 and Test Bench 2, that cost is manageable and does not prevent the scheduler from exhibiting the expected behavior. In Test Bench 3, however, the task set has no theoretical slack. That means even small implementation overheads are enough to push the last task past its deadline. The measured behavior therefore illustrates the practical difference between theoretical schedulability and real implementation overhead on embedded hardware.

Limitations and Possible Improvements

The current implementation meets the project goals, but it still has practical limitations. The monitor snapshots are not truly atomic because they are gathered through three separate getter calls. The cloned-list mechanism adds measurable overhead because it allocates and frees memory for every node in

every list at each sample. The workload model is also still an approximation. Although it was calibrated on the hardware and gave good results, it is still based on a busy loop rather than on direct measurement of true executed time.

The use of dynamic task creation and deletion per release is conceptually clean, but it also adds heap overhead and allocation pressure. In a larger system or a system with much heavier task traffic, a different strategy might be preferable. Similarly, the monitor overhead that was acceptable in TB1 and TB2 becomes much more significant in TB3, showing that debug and measurement support must be treated carefully in edge-case real-time workloads.

Several improvements would be worth considering in a future revision. One improvement would be to replace cloned-list monitoring with a single atomic snapshot message that returns counts directly. Another would be to add a lightweight debug mode versus benchmark mode switch so that prints and monitor activity can be disabled cleanly for timing runs. A more advanced improvement would be to measure executed time directly using a hardware timer or runtime trace hooks rather than relying on a calibrated busy loop. Finally, the aperiodic-task support that is currently only accommodated structurally could be extended into a fully implemented release path.

Summary

This project successfully implemented a Deadline-Driven Scheduler on top of FreeRTOS using an EDF policy enforced through dynamic priority changes. The DDS owns the task metadata and list state, the generators create jobs according to the benchmark specifications, the user tasks simulate workload and report completion, and FreeRTOS performs the actual preemptive execution.

The final system achieved the expected outcomes for the course benchmarks. Test Bench 1 behaved as a schedulable EDF task set with no overdue tasks. Test Bench 2 behaved as a lightly overloaded task set with the expected overdue behavior. Test Bench 3 demonstrated the effect of real implementation overhead on a zero-slack workload and provided a useful illustration of the difference between theoretical schedulability and practical embedded execution.

More importantly, the final design is understandable and reproducible. It uses the required FreeRTOS mechanisms appropriately, keeps scheduler ownership centralized, and demonstrates a complete DDS implementation that aligns with the project objectives.

Appendices

Appendix A: Code

Main.c

```
#include <stdio.h>
```

```

#include "stm32f4xx.h"

#include "FreeRTOS.h"
#include "task.h"

#include "../Utilities/STM32F4-Discovery/stm32f4_discovery.h"

#include "dds.h"
#include "generator.h"
#include "monitor.h"
#include "testbench.h"
#include "user_task.h"

static void App_InitBoard(void);

int main(void)
{
    const testbench_config_t *testbench_config;

    /* Set LEDs to a known state before any tasks start running. */
    App_InitBoard();

    testbench_config =
testbench_get_config((testbench_id_t)DDS_SELECTED_TESTBENCH);
    configASSERT(testbench_config != NULL);

    /* Convert benchmark execution times into calibrated busy-work
budgets. */
    testbench_calibrate_workload();

    printf(
        "DDS bench=%u hyper=%u\r\n",
        (unsigned int)DDS_SELECTED_TESTBENCH,
        (unsigned int)testbench_config->hyperperiod_ms);

    configASSERT(dds_init() == pdPASS);
    configASSERT(generator_init(testbench_config) == pdPASS);

```

```

configASSERT(monitor_init() == pdPASS);

/* From here on, FreeRTOS owns execution. */
vTaskStartScheduler();

taskDISABLE_INTERRUPTS();
for (;;)
{
}

static void App_InitBoard(void)
{
    /* LED3-LED5 are user-task indicators, LED6 is a fault indicator. */
    STM_EVAL_LEDInit(LED3);
    STM_EVAL_LEDInit(LED4);
    STM_EVAL_LEDInit(LED5);
    STM_EVAL_LEDInit(LED6);

    user_task_clear_debug_leds();
    STM_EVAL_LEDOff(LED6);
}

void vApplicationMallocFailedHook(void)
{
    /* Stop in a visible fault state if heap allocation fails. */
    STM_EVAL_LEDOn(LED6);
    taskDISABLE_INTERRUPTS();

    for (;;)
    {
    }
}

void vApplicationStackOverflowHook(TaskHandle_t task_handle, char
*task_name)
{
    (void)task_handle;
    (void)task_name;
}

```



```

    /* Stop in a visible fault state if any task overflows its stack. */
    STM_EVAL_LEDOn(LED6);
    taskDISABLE_INTERRUPTS();

    for (;;)
    {
    }
}

void vApplicationIdleHook(void)
{
}

```

Dds.c

```

#include "dds.h"

#include <stddef.h>

#include "FreeRTOS.h"
#include "queue.h"
#include "task.h"
#include "timers.h"

#include "dds_list.h"
#include "user_task.h"

#define DDS_COMMAND_QUEUE_LENGTH      ( 16U )
#define DDS_REPLY_QUEUE_LENGTH        ( 1U )
#define DDS_TASK_STACK_WORDS          ( configMINIMAL_STACK_SIZE * 2U )

static TaskHandle_t g_dds_task_handle = NULL;
static QueueHandle_t g_dds_command_queue = NULL;
static QueueHandle_t g_dds_reply_queue = NULL;
static TimerHandle_t g_dds_deadline_timer = NULL;

static dd_task_node_t *g_active_list_head = NULL;
static dd_task_node_t *g_completed_list_head = NULL;
static dd_task_node_t *g_overdue_list_head = NULL;

static uint32_t g_active_count = 0U;

```

```

static uint32_t g_completed_count = 0U;
static uint32_t g_overdue_count = 0U;

static void DDS_Task(void *pvParameters);
static void DDS_DeadlineTimerCallback(TimerHandle_t timer_handle);

static void dds_handle_release(const dd_task_t *released_task);
static void dds_handle_completion(uint32_t task_id, TickType_t
observed_completion_time);
static void dds_handle_deadline_expired(void);
static void dds_handle_get_active(void);
static void dds_handle_get_completed(void);
static void dds_handle_get_overdue(void);
static void dds_sweep_overdue(TickType_t now);
static void dds_apply_edf_priorities(void);
static void dds_rearm_deadline_timer(void);
static void dds_delete_freertos_task(TaskHandle_t task_handle);
static void dds_send_reply(dd_task_node_t *list_head);
static void dds_wake_scheduler(void);

BaseType_t dds_init(void)
{
    BaseType_t status;

    /* All scheduler events are serialized through this queue. */
    g_dds_command_queue = xQueueCreate(DDS_COMMAND_QUEUE_LENGTH,
sizeof(dds_message_t));
    if (g_dds_command_queue == NULL)
    {
        return pdFAIL;
    }

    /* Getter calls receive cloned list snapshots through this queue. */
    g_dds_reply_queue = xQueueCreate(DDS_REPLY_QUEUE_LENGTH,
sizeof(dd_task_node_t *));
    if (g_dds_reply_queue == NULL)
    {
        return pdFAIL;
    }
}

```

```

vQueueAddToRegistry(g_dds_command_queue, "QDDSCMD");
vQueueAddToRegistry(g_dds_reply_queue, "QDDSRPL");

/* This one-shot timer is always aimed at the earliest active
deadline. */
g_dds_deadline_timer = xTimerCreate(
    "DDSDL",
    pdMS_TO_TICKS(1U),
    pdFALSE,
    NULL,
    DDS_DeadlineTimerCallback);
if (g_dds_deadline_timer == NULL)
{
    return pdFAIL;
}

status = xTaskCreate(
    DDS_Task,
    "DDS",
    DDS_TASK_STACK_WORDS,
    NULL,
    DDS_PRIO_DDS,
    &g_dds_task_handle);

return status;
}

void create_dd_task(
    TaskHandle_t t_handle,
    task_type_t type,
    uint32_t task_id,
    uint32_t absolute_deadline)
{
    dds_message_t message;

    configASSERT(g_dds_command_queue != NULL);
    configASSERT(g_dds_task_handle != NULL);

    /* DDS stamps the true release time when it services this message. */
    message.type = DDS_MSG_RELEASE;

```

```

    message.data.release_task.t_handle = t_handle;
    message.data.release_task.type = type;
    message.data.release_task.task_id = task_id;
    message.data.release_task.release_time = 0U;
    message.data.release_task.absolute_deadline =
(TickType_t)absolute_deadline;
    message.data.release_task.completion_time = 0U;
    message.data.release_task.execution_time_ms = 0U;
    message.data.release_task.source_task_number = task_id >> 16;
    message.data.release_task.sequence_number = task_id & 0xFFFFU;
    message.data.release_task.state = DD_ACTIVE;

    configASSERT(xQueueSend(g_dds_command_queue, &message, portMAX_DELAY)
== pdPASS);
    dds_wake_scheduler();
}

void delete_dd_task(uint32_t task_id)
{
    dds_message_t message;

    configASSERT(g_dds_command_queue != NULL);
    configASSERT(g_dds_task_handle != NULL);

    /* Capture the observed finish time before DDS queue latency is
introduced. */
    message.type = DDS_MSG_COMPLETE;
    message.data.completion.task_id = task_id;
    message.data.completion.observed_completion_time =
xTaskGetTickCount();

    configASSERT(xQueueSend(g_dds_command_queue, &message, portMAX_DELAY)
== pdPASS);
    dds_wake_scheduler();
}

dd_task_node_t *get_active_dd_task_list(void)
{
    dds_message_t message;
    dd_task_node_t *reply;

```

```

    reply = NULL;
    message.type = DDS_MSG_GET_ACTIVE;

    configASSERT(xQueueSend(g_dds_command_queue, &message, portMAX_DELAY)
== pdPASS);
    dds_wake_scheduler();
    configASSERT(xQueueReceive(g_dds_reply_queue, &reply, portMAX_DELAY)
== pdPASS);

    return reply;
}

dd_task_node_t *get_complete_dd_task_list(void)
{
    dds_message_t message;
    dd_task_node_t *reply;

    reply = NULL;
    message.type = DDS_MSG_GET_COMPLETED;

    configASSERT(xQueueSend(g_dds_command_queue, &message, portMAX_DELAY)
== pdPASS);
    dds_wake_scheduler();
    configASSERT(xQueueReceive(g_dds_reply_queue, &reply, portMAX_DELAY)
== pdPASS);

    return reply;
}

dd_task_node_t *get_overdue_dd_task_list(void)
{
    dds_message_t message;
    dd_task_node_t *reply;

    reply = NULL;
    message.type = DDS_MSG_GET_OVERDUE;

    configASSERT(xQueueSend(g_dds_command_queue, &message, portMAX_DELAY)
== pdPASS);

```

```

        dds_wake_scheduler();
        configASSERT(xQueueReceive(g_dds_reply_queue, &reply, portMAX_DELAY)
== pdPASS);

        return reply;
    }

static void DDS_Task(void *pvParameters)
{
    dds_message_t message;

    (void)pvParameters;

    for (;;)
    {
        /* DDS sleeps until an interface call or timer event wakes it. */
        vTaskSuspend(NULL);

        /* Drain the whole queue so related events are handled as one
batch. */
        while (xQueueReceive(g_dds_command_queue, &message, 0U) == pdPASS)
        {
            switch (message.type)
            {
                case DDS_MSG_RELEASE:
                    dds_handle_release(&message.data.release_task);
                    break;

                case DDS_MSG_COMPLETE:
                    dds_handle_completion(
                        message.data.completion.task_id,
                        message.data.completion.observed_completion_time);
                    break;

                case DDS_MSG_GET_ACTIVE:
                    dds_handle_get_active();
                    break;

                case DDS_MSG_GET_COMPLETED:
                    dds_handle_get_completed();

```

```

        break;

        case DDS_MSG_GET_OVERDUE:
            dds_handle_get_overdue();
            break;

        case DDS_MSG_DEADLINE_EXPIRED:
            dds_handle_deadline_expired();
            break;

        default:
            configASSERT(0);
            break;
    }
}

/* Recompute scheduler state after all queued events are applied.
*/
dds_sweep_overdue(xTaskGetTickCount());
dds_apply_edf_priorities();
dds_rearm_deadline_timer();
}
}

static void dds_handle_release(const dd_task_t *released_task)
{
    dd_task_t task_record;
    dd_task_node_t *node;

    configASSERT(released_task != NULL);

    task_record = *released_task;
    task_record.release_time = xTaskGetTickCount();
    task_record.completion_time = 0U;
    task_record.state = DD_ACTIVE;

    /* Active-list ordering is maintained by sorted insertion. */
    node = dd_list_create_node(&task_record);
    configASSERT(node != NULL);

```

```

        dd_list_insert_sorted(&g_active_list_head, node);
        g_active_count++;
    }

static void dds_handle_completion(uint32_t task_id, TickType_t
observed_completion_time)
{
    dd_task_node_t *node;

    node = dd_list_remove_by_id(&g_active_list_head, task_id);
    if (node == NULL)
    {
        return;
    }

    g_active_count--;

    node->task.completion_time = observed_completion_time;
    user_task_set_debug_led(node->task.source_task_number, pdFALSE);

    /* Completion on the exact deadline still counts as on time. */
    if (observed_completion_time <= node->task.absolute_deadline)
    {
        node->task.state = DD_COMPLETED;
        dd_list_append_node(&g_completed_list_head, node);
        g_completed_count++;
    }
    else
    {
        node->task.state = DD_OVERDUE;
        dd_list_append_node(&g_overdue_list_head, node);
        g_overdue_count++;
    }

    dds_delete_freertos_task(node->task.t_handle);
    node->task.t_handle = NULL;
}

static void dds_handle_deadline_expired(void)
{

```



```

    /* The actual overdue handling happens in the common sweep step. */
}

static void dds_handle_get_active(void)
{
    dd_task_node_t *clone;

    clone = dd_list_clone(g_active_list_head);
    configASSERT((g_active_list_head == NULL) || (clone != NULL));
    dds_send_reply(clone);
}

static void dds_handle_get_completed(void)
{
    dd_task_node_t *clone;

    clone = dd_list_clone(g_completed_list_head);
    configASSERT((g_completed_list_head == NULL) || (clone != NULL));
    dds_send_reply(clone);
}

static void dds_handle_get_overdue(void)
{
    dd_task_node_t *clone;

    clone = dd_list_clone(g_overdue_list_head);
    configASSERT((g_overdue_list_head == NULL) || (clone != NULL));
    dds_send_reply(clone);
}

static void dds_sweep_overdue(TickType_t now)
{
    dd_task_node_t *node;

    /* Only the head needs checking because the active list is
    deadline-sorted. */
    while ((g_active_list_head != NULL) &&
           (now > g_active_list_head->task.absolute_deadline))
    {
        node = g_active_list_head;

```

```

    g_active_list_head = node->next;
    node->next = NULL;

    g_active_count--;
    node->task.state = DD_OVERDUE;

    user_task_set_debug_led(node->task.source_task_number, pdFALSE);
    dd_list_append_node(&g_overdue_list_head, node);
    g_overdue_count++;

    dds_delete_freertos_task(node->task.t_handle);
    node->task.t_handle = NULL;
}
}

static void dds_apply_edf_priorities(void)
{
    dd_task_node_t *node;

    user_task_clear_debug_leds();

    /* Promote the earliest-deadline task and demote the rest. */
    node = g_active_list_head;
    while (node != NULL)
    {
        configASSERT(node->task.t_handle != NULL);

        if (node == g_active_list_head)
        {
            vTaskPrioritySet(node->task.t_handle, DDS_PRIO_USER_HIGH);
        }
        else
        {
            vTaskPrioritySet(node->task.t_handle, DDS_PRIO_USER_LOW);
        }

        node = node->next;
    }

    if (g_active_list_head != NULL)

```

```

    {

user_task_set_debug_led(g_active_list_head->task.source_task_number,
pdTRUE);

        vTaskResume(g_active_list_head->task.t_handle);

    }
}

static void dds_rearm_deadline_timer(void)
{
    TickType_t now;
    TickType_t delay;

    if (g_active_list_head == NULL)
    {
        configASSERT(xTimerStop(g_dds_deadline_timer, 0U) == pdPASS);
        return;
    }

    /* Rearm the timer to the next deadline instead of polling
continuously. */
    now = xTaskGetTickCount();
    if (g_active_list_head->task.absolute_deadline > now)
    {
        delay = g_active_list_head->task.absolute_deadline - now;
    }
    else
    {
        delay = 1U;
    }

    configASSERT(xTimerChangePeriod(g_dds_deadline_timer, delay, 0U) ==
pdPASS);
}

static void dds_delete_freertos_task(TaskHandle_t task_handle)
{
    if (task_handle != NULL)
    {
        vTaskDelete(task_handle);
    }
}

```

```

    }
}

static void dds_send_reply(dd_task_node_t *list_head)
{
    configASSERT(xQueueSend(g_dds_reply_queue, &list_head, portMAX_DELAY)
== pdPASS);
}

static void dds_wake_scheduler(void)
{
    vTaskResume(g_dds_task_handle);
}

static void DDS_DeadlineTimerCallback(TimerHandle_t timer_handle)
{
    dds_message_t message;
    BaseType_t send_status;

    (void)timer_handle;

    /* Keep the callback short; DDS does the real work in task context. */
    message.type = DDS_MSG_DEADLINE_EXPIRED;
    send_status = xQueueSend(g_dds_command_queue, &message, 0U);
    configASSERT(send_status == pdPASS);

    dds_wake_scheduler();
}

```

Generator.c

```

#include "generator.h"

#include "FreeRTOS.h"
#include "queue.h"
#include "task.h"
#include "timers.h"

#include "dds.h"
#include "dds_types.h"
#include "testbench.h"

```

```

#include "user_task.h"

#define GENERATOR_COUNT ( 3U )
#define GENERATOR_QUEUE_LENGTH ( 4U )
#define GENERATOR_TASK_STACK_WORDS ( configMINIMAL_STACK_SIZE * 2U )

typedef struct
{
    uint32_t source_task_number;
    uint32_t execution_time_ms;
    uint32_t period_ms;
    uint32_t sequence_number;
    task_type_t type;
    QueueHandle_t release_queue;
    TaskHandle_t task_handle;
    TimerHandle_t timer_handle;
    const char *task_name;
    const char *timer_name;
    const char *queue_name;
} generator_context_t;

/* One context per periodic source task. */
static generator_context_t g_generators[GENERATOR_COUNT];

static void Generator_Task(void *pvParameters);
static void GeneratorTimerCallback(TimerHandle_t timer_handle);
static void generator_release_job(generator_context_t *context, TickType_t release_tick);

static uint32_t generator_make_task_id(uint32_t source_task_number,
uint32_t sequence_number)
{
    /* Upper 16 bits identify the source, lower 16 bits identify the
release. */
    return (source_task_number << 16) | (sequence_number & 0xFFFFU);
}

BaseType_t generator_init(const testbench_config_t *config)
{
    static const char * const generator_task_names[GENERATOR_COUNT] =

```

```

{
    "GEN1",
    "GEN2",
    "GEN3"
};

static const char * const generator_timer_names[GENERATOR_COUNT] =
{
    "T1REL",
    "T2REL",
    "T3REL"
};

static const char * const generator_queue_names[GENERATOR_COUNT] =
{
    "QGEN1",
    "QGEN2",
    "QGEN3"
};

BaseType_t status;
TickType_t initial_release_tick;
uint32_t index;

configASSERT(config != NULL);

initial_release_tick = 0U;

for (index = 0U; index < GENERATOR_COUNT; index++)
{
    generator_context_t *context = &g_generators[index];

    context->source_task_number = index + 1U;
    context->execution_time_ms = config->exec_ms[index];
    context->period_ms = config->period_ms[index];
    context->sequence_number = 0U;
    context->type = PERIODIC;
    context->task_name = generator_task_names[index];
    context->timer_name = generator_timer_names[index];
    context->queue_name = generator_queue_names[index];

```

```

    /* Queue release ticks so multiple timer expiries are never
collapsed. */
    context->release_queue = xQueueCreate(GENERATOR_QUEUE_LENGTH,
sizeof(TickType_t));
    if (context->release_queue == NULL)
    {
        return pdFAIL;
    }

    vQueueAddToRegistry(context->release_queue, context->queue_name);

    status = xTaskCreate(
        Generator_Task,
        context->task_name,
        GENERATOR_TASK_STACK_WORDS,
        context,
        DDS_PRIO_GENERATOR,
        &context->task_handle);
    if (status != pdPASS)
    {
        return pdFAIL;
    }

    /* Each source has its own periodic release timer. */
    context->timer_handle = xTimerCreate(
        context->timer_name,
        pdMS_TO_TICKS(context->period_ms),
        pdTRUE,
        context,
        GeneratorTimerCallback);
    if (context->timer_handle == NULL)
    {
        return pdFAIL;
    }
}

for (index = 0U; index < GENERATOR_COUNT; index++)
{
    configASSERT(xTimerStart(g_generators[index].timer_handle, 0U) ==
pdPASS);
}

```

```

        /* Seed the initial t=0 release before the periodic timer takes
over. */
        configASSERT(xQueueSend(g_generators[index].release_queue,
&initial_release_tick, 0U) == pdPASS);
    }

    return pdPASS;
}

static void Generator_Task(void *pvParameters)
{
    TickType_t release_tick;
    generator_context_t *context;

    context = (generator_context_t *)pvParameters;
    configASSERT(context != NULL);

    for (;;)
    {
        if (uxQueueMessagesWaiting(context->release_queue) == 0U)
        {
            vTaskSuspend(NULL);
            continue;
        }

        /* Drain all queued release events before suspending again. */
        while (xQueueReceive(context->release_queue, &release_tick, 0U) ==
pdPASS)
        {
            context->sequence_number++;
            generator_release_job(context, release_tick);
        }
    }
}

static void generator_release_job(generator_context_t *context, TickType_t
release_tick)
{
    static const char * const user_task_names[GENERATOR_COUNT] =
    {

```



```

        "USR1",
        "USR2",
        "USR3"
    };

    TickType_t absolute_deadline;
    user_task_params_t *params;
    TaskHandle_t user_task_handle;
    BaseType_t status;
    uint32_t task_id;

    configASSERT(context != NULL);

    task_id = generator_make_task_id(context->source_task_number,
context->sequence_number);

    /* Each release gets its own user-task parameter block. */
    params = (user_task_params_t *)pvPortMalloc(sizeof(*params));
    configASSERT(params != NULL);

    params->task_id = task_id;
    params->execution_time_ms = context->execution_time_ms;
    params->source_task_number = context->source_task_number;
    params->work_iterations =
testbench_get_work_iterations(context->execution_time_ms);

    user_task_handle = NULL;
    status = xTaskCreate(
        User_Task,
        user_task_names[context->source_task_number - 1U],
        configMINIMAL_STACK_SIZE,
        params,
        DDS_PRIO_USER_LOW,
        &user_task_handle);

    if (status != pdPASS)
    {
        vPortFree(params);
        configASSERT(0);
    }

```

```

    /* DDS decides when the job is allowed to start running. */
    vTaskSuspend(user_task_handle);

    absolute_deadline = release_tick + pdMS_TO_TICKS(context->period_ms);
    create_dd_task(user_task_handle, context->type, task_id,
(uint32_t)absolute_deadline);
}

static void GeneratorTimerCallback(TimerHandle_t timer_handle)
{
    BaseType_t send_status;
    TickType_t release_tick;
    generator_context_t *context;

    context = (generator_context_t *)pvTimerGetTimerID(timer_handle);
    configASSERT(context != NULL);

    /* Queue the release time before waking the generator task. */
    release_tick = xTaskGetTickCount();
    send_status = xQueueSend(context->release_queue, &release_tick, 0U);
    configASSERT(send_status == pdPASS);

    vTaskResume(context->task_handle);
}

```

Dds_list.c

```

#include "dds_list.h"

#include <stddef.h>

#include "FreeRTOS.h"

static int dd_task_precedes(const dd_task_t *lhs, const dd_task_t *rhs)
{
    /* Deterministic EDF tie-break: deadline, then release time, then task
ID. */
    if (lhs->absolute_deadline < rhs->absolute_deadline)
    {

```

```

        return 1;
    }

    if (lhs->absolute_deadline > rhs->absolute_deadline)
    {
        return 0;
    }

    if (lhs->release_time < rhs->release_time)
    {
        return 1;
    }

    if (lhs->release_time > rhs->release_time)
    {
        return 0;
    }

    return lhs->task_id < rhs->task_id;
}

dd_task_node_t *dd_list_create_node(const dd_task_t *task)
{
    dd_task_node_t *node;

    /* DDS owns list nodes explicitly, so each one is heap-allocated. */
    node = (dd_task_node_t *)pvPortMalloc(sizeof(*node));
    if (node == NULL)
    {
        return NULL;
    }

    node->task = *task;
    node->next = NULL;
    return node;
}

void dd_list_insert_sorted(dd_task_node_t **head, dd_task_node_t *node)
{
    dd_task_node_t *current;

```

```

configASSERT(head != NULL);
configASSERT(node != NULL);

if ((*head == NULL) || dd_task_precedes(&node->task, &(*head)->task))
{
    node->next = *head;
    *head = node;
    return;
}

/* Walk until the first node that should come after the new task. */
current = *head;
while ((current->next != NULL) &&
        !dd_task_precedes(&node->task, &current->next->task))
{
    current = current->next;
}

node->next = current->next;
current->next = node;
}

dd_task_node_t *dd_list_remove_by_id(dd_task_node_t **head, uint32_t
task_id)
{
    dd_task_node_t *current;
    dd_task_node_t *previous;

    configASSERT(head != NULL);

    previous = NULL;
    current = *head;

    while (current != NULL)
    {
        if (current->task.task_id == task_id)
        {
            if (previous == NULL)
            {

```

```

        *head = current->next;
    }
    else
    {
        previous->next = current->next;
    }

    current->next = NULL;
    return current;
}

previous = current;
current = current->next;
}

return NULL;
}

void dd_list_append_node(dd_task_node_t **head, dd_task_node_t *node)
{
    dd_task_node_t *current;

    configASSERT(head != NULL);
    configASSERT(node != NULL);

    node->next = NULL;

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    /* Completed and overdue lists preserve arrival order, not deadline
order. */
    current = *head;
    while (current->next != NULL)
    {
        current = current->next;
    }

```

```

        current->next = node;
    }

dd_task_node_t *dd_list_clone(const dd_task_node_t *head)
{
    dd_task_node_t *clone_head;
    dd_task_node_t *clone_node;

    /* Getter calls return snapshots so callers never touch DDS-owned
lists. */
    clone_head = NULL;

    while (head != NULL)
    {
        clone_node = dd_list_create_node(&head->task);
        if (clone_node == NULL)
        {
            dd_list_free(clone_head);
            return NULL;
        }

        dd_list_append_node(&clone_head, clone_node);
        head = head->next;
    }

    return clone_head;
}

void dd_list_free(dd_task_node_t *head)
{
    dd_task_node_t *next;

    while (head != NULL)
    {
        next = head->next;
        vPortFree(head);
        head = next;
    }
}

```

Monitor.c

```
#include "monitor.h"

#include <stdio.h>

#include "FreeRTOS.h"
#include "queue.h"
#include "task.h"
#include "timers.h"

#include "dds.h"
#include "dds_types.h"

#define MONITOR_EVENT_QUEUE_LENGTH      ( 2U )
#define MONITOR_TASK_STACK_WORDS       ( configMINIMAL_STACK_SIZE * 2U )
#define MONITOR_PERIOD_MS               ( 500U )

static QueueHandle_t g_monitor_event_queue = NULL;
static TaskHandle_t g_monitor_task_handle = NULL;
static TimerHandle_t g_monitor_timer_handle = NULL;

static void Monitor_Task(void *pvParameters);
static void MonitorTimerCallback(TimerHandle_t timer_handle);
static uint32_t monitor_count_list(const dd_task_node_t *head);
static void monitor_free_list(dd_task_node_t *head);

BaseType_t monitor_init(void)
{
    BaseType_t status;

    /* Monitor wake events are queued so timer expiries are not lost. */
    g_monitor_event_queue = xQueueCreate(MONITOR_EVENT_QUEUE_LENGTH,
sizeof(TickType_t));
    if (g_monitor_event_queue == NULL)
    {
        return pdFAIL;
    }

    vQueueAddToRegistry(g_monitor_event_queue, "QMON");
```

```

    status = xTaskCreate(
        Monitor_Task,
        "MON",
        MONITOR_TASK_STACK_WORDS,
        NULL,
        DDS_PRIO_MONITOR,
        &g_monitor_task_handle);
    if (status != pdPASS)
    {
        return pdFAIL;
    }

    g_monitor_timer_handle = xTimerCreate(
        "MON",
        pdMS_TO_TICKS(MONITOR_PERIOD_MS),
        pdTRUE,
        NULL,
        MonitorTimerCallback);
    if (g_monitor_timer_handle == NULL)
    {
        return pdFAIL;
    }

    configASSERT(xTimerStart(g_monitor_timer_handle, 0U) == pdPASS);
    return pdPASS;
}

static void Monitor_Task(void *pvParameters)
{
    TickType_t token;
    TickType_t sample_tick;
    dd_task_node_t *active;
    dd_task_node_t *complete;
    dd_task_node_t *overdue;

    (void)pvParameters;

    for (;;)
    {

```



```

    if (uxQueueMessagesWaiting(g_monitor_event_queue) == 0U)
    {
        vTaskSuspend(NULL);
        continue;
    }

    /* Collapse multiple wake tokens into one sample at the newest
tick. */
    sample_tick = xTaskGetTickCount();
    while (xQueueReceive(g_monitor_event_queue, &token, 0U) == pdPASS)
    {
        sample_tick = token;
    }

    /* Getter calls return clones, so free them after sampling. */
    active = get_active_dd_task_list();
    complete = get_complete_dd_task_list();
    overdue = get_overdue_dd_task_list();

    (void)sample_tick;
    /*
printf(
    "MON t=%u A=%u C=%u O=%u\r\n",
    (unsigned int)sample_tick,
    (unsigned int)monitor_count_list(active),
    (unsigned int)monitor_count_list(complete),
    (unsigned int)monitor_count_list(overdue));
*/

    monitor_free_list(active);
    monitor_free_list(complete);
    monitor_free_list(overdue);
}
}

static void MonitorTimerCallback(TimerHandle_t timer_handle)
{
    BaseType_t send_status;
    TickType_t token;

```

```

        (void)timer_handle;

        /* Keep callbacks short and wake the monitor task to do the heavier
work. */
        token = xTaskGetTickCount();
        send_status = xQueueSend(g_monitor_event_queue, &token, 0U);
        configASSERT(send_status == pdPASS);

        vTaskResume(g_monitor_task_handle);
    }

static uint32_t monitor_count_list(const dd_task_node_t *head)
{
    uint32_t count;

    count = 0U;
    while (head != NULL)
    {
        count++;
        head = head->next;
    }

    return count;
}

static void monitor_free_list(dd_task_node_t *head)
{
    dd_task_node_t *next;

    while (head != NULL)
    {
        next = head->next;
        vPortFree(head);
        head = next;
    }
}

```

Testbench.c

```
#include "testbench.h"
```

```
#include <limits.h>

#include "stm32f4xx.h"

#include "FreeRTOS.h"

#define TESTBENCH_ESTIMATED_CYCLES_PER_ITERATION    ( 21UL )

extern uint32_t SystemCoreClock;

static const testbench_config_t g_testbench_1 =
{
    { 95U, 150U, 250U },
    { 500U, 500U, 750U },
    1500U
};

static const testbench_config_t g_testbench_2 =
{
    { 95U, 150U, 250U },
    { 250U, 500U, 750U },
    1500U
};

static const testbench_config_t g_testbench_3 =
{
    { 100U, 200U, 200U },
    { 500U, 500U, 500U },
    500U
};

static uint32_t g_work_iterations_per_ms = 1U;

const testbench_config_t *testbench_get_config(testbench_id_t
testbench_id)
{
    switch (testbench_id)
    {
        case TESTBENCH_1:
            return &g_testbench_1;
    }
}
```

```

        case TESTBENCH_2:
            return &g_testbench_2;

        case TESTBENCH_3:
            return &g_testbench_3;

        default:
            return NULL;
    }
}

void testbench_calibrate_workload(void)
{
    uint64_t iterations_per_ms;

    /* Convert the estimated loop cost into a per-millisecond work budget.
    */
    iterations_per_ms =
        (uint64_t)SystemCoreClock /
        (1000ULL * (uint64_t)TESTBENCH_ESTIMATED_CYCLES_PER_ITERATION);

    if (iterations_per_ms == 0ULL)
    {
        iterations_per_ms = 1ULL;
    }

    if (iterations_per_ms > UINT_MAX)
    {
        iterations_per_ms = UINT_MAX;
    }

    g_work_iterations_per_ms = (uint32_t)iterations_per_ms;
}

uint32_t testbench_get_work_iterations(uint32_t execution_time_ms)
{
    uint64_t iterations;

```

```

    /* Scale the calibrated per-ms budget to the requested execution time.
    */
    iterations = (uint64_t)execution_time_ms *
(uint64_t)g_work_iterations_per_ms;
    if (iterations == 0ULL)
    {
        iterations = 1ULL;
    }

    if (iterations > UINT_MAX)
    {
        iterations = UINT_MAX;
    }

    return (uint32_t)iterations;
}

uint32_t testbench_get_estimated_cycles_per_iteration(void)
{
    return TESTBENCH_ESTIMATED_CYCLES_PER_ITERATION;
}

uint32_t testbench_get_suggested_cycles_per_iteration(
    uint32_t target_execution_ms,
    TickType_t observed_elapsed_ticks)
{
    uint64_t observed_ms;
    uint64_t suggested;

    if ((target_execution_ms == 0U) || (observed_elapsed_ticks == 0U))
    {
        return TESTBENCH_ESTIMATED_CYCLES_PER_ITERATION;
    }

    observed_ms =
        ((uint64_t)observed_elapsed_ticks * 1000ULL) /
        (uint64_t)configTICK_RATE_HZ;

    if (observed_ms == 0ULL)
    {

```

```

        observed_ms = 1ULL;
    }

    /* This helper was used during calibration to retune the loop
estimate. */
    suggested =
        ((uint64_t)TESTBENCH_ESTIMATED_CYCLES_PER_ITERATION * observed_ms)
/
        (uint64_t)target_execution_ms;

    if (suggested == 0ULL)
    {
        suggested = 1ULL;
    }

    if (suggested > UINT_MAX)
    {
        suggested = UINT_MAX;
    }

    return (uint32_t)suggested;
}

```

User_task.c

```

#include "user_task.h"

#include "stm32f4xx.h"

#include "FreeRTOS.h"
#include "dds.h"
#include "dds_types.h"

#include "../Utilities/STM32F4-Discovery/stm32f4_discovery.h"

static volatile uint32_t g_user_task_sink = 0U;

static Led_TypeDef user_task_get_led(uint32_t source_task_number,
BaseType_t *valid)
{

```

```

switch (source_task_number)
{
    case 1U:
        *valid = pdTRUE;
        return LED3;

    case 2U:
        *valid = pdTRUE;
        return LED4;

    case 3U:
        *valid = pdTRUE;
        return LED5;

    default:
        *valid = pdFALSE;
        return LED3;
}
}

static void busy_work_for_budget(uint32_t work_iterations)
{
    volatile uint32_t accumulator = g_user_task_sink;
    uint32_t index;

    /* Burn CPU in a way the compiler cannot trivially optimize away. */
    for (index = 0U; index < work_iterations; index++)
    {
        accumulator ^= (index + 0x9E3779B9UL);
        __NOP();
    }

    g_user_task_sink = accumulator;
}

void user_task_set_debug_led(uint32_t source_task_number, BaseType_t
is_on)
{
    BaseType_t valid;
    Led_TypeDef led;

```

```

    led = user_task_get_led(source_task_number, &valid);
    if (valid == pdFALSE)
    {
        return;
    }

    if (is_on != pdFALSE)
    {
        STM_EVAL_LEDOn(led);
    }
    else
    {
        STM_EVAL_LEDOff(led);
    }
}

void user_task_clear_debug_leds(void)
{
    STM_EVAL_LEDOff(LED3);
    STM_EVAL_LEDOff(LED4);
    STM_EVAL_LEDOff(LED5);
}

void User_Task(void *pvParameters)
{
    uint32_t source_task_number;
    uint32_t task_id;
    uint32_t work_iterations;
    user_task_params_t *params;

    params = (user_task_params_t *)pvParameters;
    configASSERT(params != NULL);

    /* Copy what is needed locally so the parameter block can be freed
early. */
    task_id = params->task_id;
    source_task_number = params->source_task_number;
    work_iterations = params->work_iterations;
    vPortFree(params);

```



```

    /* LEDs provide a quick visual trace of which source task is
executing. */
    user_task_set_debug_led(source_task_number, pdTRUE);
    busy_work_for_budget(work_iterations);
    user_task_set_debug_led(source_task_number, pdFALSE);

    /* DDS performs the actual state transition and task deletion. */
    delete_dd_task(task_id);
    vTaskSuspend(NULL);
}

```

Appendix B: Design Document

System Overview

The system will contain one DDS task, three generator tasks, dynamically created user tasks, one monitor task, several queues, and several software timers.

Planned Components

The DDS task will be the highest-priority task in the system. It will own the active, completed, and overdue DD-task lists and will wake only when a release, completion, monitor request, or deadline event occurs.

There will be three periodic generator tasks, one for each benchmark task source. Each generator will have its own timer and queue. When a release timer expires, the matching generator will create a user task and register it with the DDS.

User tasks will be ordinary FreeRTOS tasks that simulate CPU work with a busy loop. When a user task finishes, it will call `delete_dd_task(task_id)` and suspend itself so the DDS can delete it safely.

The monitor task will periodically request the active, completed, and overdue lists from the DDS and report task counts for testing and debugging.

Queues and Timers

The design will use one DDS command queue for all scheduler requests and one DDS reply queue for list snapshots returned to the monitor. Each generator will have its own small release queue so release events are not lost if two releases happen close together. The monitor may also use a small event queue so timer wakeups are preserved.

The timers in the system will be three periodic release timers, one monitor timer, and one one-shot DDS deadline timer. The deadline timer will always be armed to the earliest active deadline.

Scheduling Behavior

The active DD-task list will always remain sorted by earliest absolute deadline. If two tasks have the same deadline, the earlier release time will be used as the tie-breaker. If release times are also equal, the smaller task ID will be used.

The DDS will enforce EDF by changing FreeRTOS task priorities. The user task at the head of the active list will be assigned a high user-task priority, and all other active user tasks will be assigned a low user-task priority. Because FreeRTOS is preemptive, the earliest-deadline task will then run first.

When a task finishes, the DDS will compare its completion time against its absolute deadline. If the task finishes on or before its deadline, it will move to the completed list. Otherwise, it will move to the overdue list.

Data Structures and Priorities

The core DDS structure will be a `dd_task_t` record containing the task handle, task ID, task type, release time, absolute deadline, completion time, and a small amount of debugging information. The active, completed, and overdue lists will be singly linked lists of these task records. The active list will use sorted insertion so EDF ordering is preserved after every release or completion.

The DDS task will have the highest priority. The timer service task will be just below it. The monitor and generator tasks will run above user tasks so they can still execute while DD work exists. The currently selected user task will be promoted to a higher user-task priority, while all other active user tasks will remain at a lower priority.